ACTIVITY GUIDE 2

CODE:

```python
class Queue:
    def __init__(self):
        self.items = []

    def is_empty(self):
        return len(self.items) == 0

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if self.is_empty():
            return "Queue is empty"
        return self.items.pop(0)

    def peek(self):
        if self.is_empty():
            return "Queue is empty"
        return self.items[0]

    def size(self):
        return len(self.items)

    def display(self):
        return self.items
```

```python

# Simulating the given sequence of operations for `queue1`
queue1 = Queue()

# Operations from queue1 as listed by the user
operations1 = [
    ('enqueue', 5), ('enqueue', 3), ('size', ),
    ('dequeue', None), ('is_empty', ), ('dequeue', None),
    ('is_empty', ), ('dequeue', None), ('enqueue', 7),
    ('enqueue', 9), ('peek', ), ('enqueue', 4),
    ('size', ), ('dequeue', None)
]
```

```python
# Perform the operations for queue1
print("QUEUE 1 (from the table)")
for operation in operations1:
    if operation[0] == 'enqueue':
        queue1.enqueue(operation[1])
        print(f"Enqueued: {operation[1]}, Queue: {queue1.display()}")
    elif operation[0] == 'dequeue':
        dequeued_value = queue1.dequeue()
        print(f"Dequeued: {dequeued_value}, Queue: {queue1.display()}")
    elif operation[0] == 'size':
        print(f"Size: {queue1.size()}")
    elif operation[0] == 'is_empty':
        print(f"Is Queue Empty?: {queue1.is_empty()}")
    elif operation[0] == 'peek':
        print(f"First element: {queue1.peek()}")
print()
print(f"Final Queue: {queue1.display()}")
```

```python
# Simulating the additional sequence of operations for another queue
queue2 = Queue()

# Operations for queue2 as provided in the new code
operations2 = [
    ('enqueue', 5), ('enqueue', 3), ('dequeue', None),
    ('enqueue', 2), ('enqueue', 8), ('dequeue', None),
    ('dequeue', None), ('enqueue', 9), ('enqueue', 1),
    ('dequeue', None), ('enqueue', 7), ('enqueue', 6),
    ('dequeue', None), ('dequeue', None), ('enqueue', 4),
    ('dequeue', None), ('dequeue', None)
```

```python

# Perform the operations for queue2
print()
print("QUEUE 2:")
for operation in operations2:
    if operation[0] == 'enqueue':
        queue2.enqueue(operation[1])
        print(f"Enqueued: {operation[1]}, Queue: {queue2.display()}")
    elif operation[0] == 'dequeue':
        dequeued_value = queue2.dequeue()
        print(f"Dequeued: {dequeued_value}, Queue: {queue2.display()}")

#display the final queue
print()
print(f"Final Queue: {queue2.display()}")
```

OUTPUT:

```
QUEUE 1 (from the table)
Enqueued: 5, Queue: [5]
Enqueued: 3, Queue: [5, 3]
Size: 2
Dequeued: 5, Queue: [3]
Is Queue Empty?: False
Dequeued: 3, Queue: []
Is Queue Empty?: True
Dequeued: Queue is empty, Queue: []
Enqueued: 7, Queue: [7]
Enqueued: 9, Queue: [7, 9]
First element: 7
Enqueued: 4, Queue: [7, 9, 4]
Size: 3
Dequeued: 7, Queue: [9, 4]

Final Queue: [9, 4]
```

```
QUEUE 2:
Enqueued: 5, Queue: [5]
Enqueued: 3, Queue: [5, 3]
Dequeued: 5, Queue: [3]
Enqueued: 2, Queue: [3, 2]
Enqueued: 8, Queue: [3, 2, 8]
Dequeued: 3, Queue: [2, 8]
Dequeued: 2, Queue: [8]
Enqueued: 9, Queue: [8, 9]
Enqueued: 1, Queue: [8, 9, 1]
Dequeued: 8, Queue: [9, 1]
Enqueued: 7, Queue: [9, 1, 7]
Enqueued: 6, Queue: [9, 1, 7, 6]
Dequeued: 9, Queue: [1, 7, 6]
Dequeued: 1, Queue: [7, 6]
Enqueued: 4, Queue: [7, 6, 4]
Dequeued: 7, Queue: [6, 4]
Dequeued: 6, Queue: [4]

Final Queue: [4]
```